

10K TPS Capacity Plan

ต้องใช้ pod และ resource เท่าไรถึงจะรับ 10,000 requests/วินาที โดยที่ทั้ง API และ database ใช้ทรัพยากรไม่เกิน 60 % ของ limit คำตอบมาจากการวัดจริงแบบขั้นบันไดที่ 1,000 ถึง 4,000 TPS แล้วคำนวณต่อ ไม่ใช่การเดา

วัดจริงถึง	REQUESTS	ล้มเหลว	P95
4,000 TPS	579,842	0	1.55 ms
เพดานที่ตั้ง	หลักฐาน		
60 %	c1-capacity		

01 คำตอบ

API 4 pods, Postgres 1 pod 6 cores / 3Gi

CONFIG ที่แนะนำสำหรับ 10,000 TPS					
ส่วนประกอบ	REPLICAS	CPU LIMIT	MEMORY LIMIT	CPU ที่ใช้จริง	MEMORY ที่ใช้จริง
API (Gin + pgx) scale out ไม่ใช่ scale up	4	500m ต่อ pod รวม 2,000m	128Mi ต่อ pod	1,099m = 55 %	18 MiB = 14 %
Postgres ตัวเดียว ไม่ต้อง replica	1	6,000m	3Gi	2,355m = 39 %	1,510 MiB = 49 %
ตั้ง requests ไว้ที่ 60 % ของ limit พอดี เพื่อให้เส้นสีเขียวในกราฟทุกใบคือเส้นงบประมาณ ถ้าเส้นข้อมูลไม่ข้ามเส้นเขียว แปลว่าผ่านเงื่อนไข					

ตัวเลขที่ต้องตั้งใน manifest ทั้งหมด

API Deployment

replicas: 4

limits: cpu 500m mem 128Mi

requests: cpu 300m mem 77Mi

env: GOMAXPROCS=1 GOMEMLIMIT=100MiB shared_buffers 640MB max_connections 100

Postgres StatefulSet

replicas: 1

limits: cpu 6000m mem 3Gi

requests: cpu 3600m mem 1843Mi

สรุปสั้นที่สุด

ฝั่ง API ให้เพิ่มจำนวน **pod** จาก 1 เป็น 4 โดยไม่ต้องขยาย pod ให้ใหญ่ขึ้น เพราะ Go ถูกตั้ง

`GOMAXPROCS=1` อยู่แล้ว การเพิ่ม CPU ให้ pod เดียวจึงไม่ช่วย ส่วนฝั่ง database ให้ขยาย **pod** เดิม

จาก 2 เป็น 6 cores และจาก 1Gi เป็น 3Gi ยังคงเป็น Postgres ตัวเดียว ไม่ต้องทำ replication

02 วิเคราะห์คำตอบ

วัดที่อัตราต่ำ แล้วคำนวณขึ้นไป ดีกว่ายิ่ง 10,000 TPS จริง

ถ้ายิ่ง 10,000 TPS จากเครื่องเดียวกับที่รัน cluster อยู่ ตัว k6 เองจะกิน CPU หลาย core ไปแย่งกับ pod ที่เรากำลังวัด ตัวเลขที่ได้จะไม่ใช่ต้นทุนจริงของระบบ แต่เป็นต้นทุนของระบบที่ถูกแย่ง CPU วิธีที่ให้คำตอบแม่นยำกว่าคือ

1. ยิ่งเป็นขั้นบันไดที่อัตราซึ่งเครื่องรับไหวสบายๆ คือ 1,000 → 2,000 → 3,000 → 4,000 TPS ขึ้นละ 60 วินาที
2. วัด CPU ที่ใช้จริงในแต่ละขั้น แล้วดูว่าต้นทุนต่อ **1 TPS** คงที่หรือไม่
3. ถ้าคงที่ แปลว่าความสัมพันธ์เป็นเส้นตรง จึงลากเส้นต่อไปที่ 10,000 TPS ได้อย่างมีเหตุผล
4. ตรวจจว่าค่าที่คำนวณได้ยังอยู่ใต้เพดาน 60 % และตรวจข้อจำกัดอื่นที่ CPU ไม่ครอบคลุมด้วย

จุดสำคัญของการออกแบบ TEST

ใช้ executor แบบ `ramping-arrival-rate` ไม่ใช้การกำหนดจำนวน VU เพราะ **VU ไม่ใช่ TPS** ถ้ากำหนด VU แล้วระบบช้าลง TPS จะลดตามเองโดยที่เราไม่รู้ตัว แต่ arrival rate คือการบังคับอัตราค่าขอต่อวินาทีโดยตรง ถ้าระบบตามไม่ทัน k6 จะรายงาน `dropped_iterations` ให้เห็นชัดเจน ในรอบนี้ยังไป 579,842 ครั้ง จากที่ควรได้ 580,000 คิดเป็น **99.97 %** แปลว่าอัตราที่ตั้งไว้เกิดขึ้นจริงเกือบสมบูรณ์

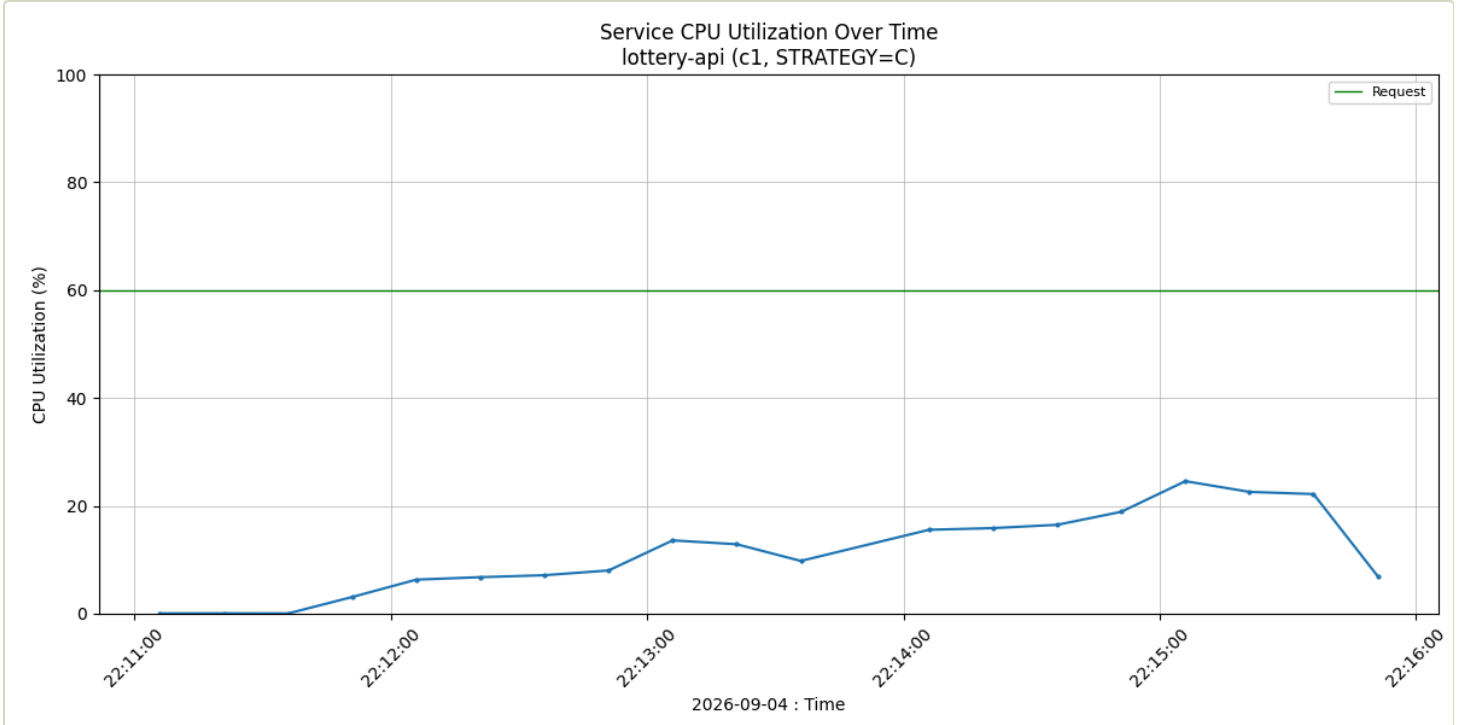
ส่วนผสมของ traffic ตั้งไว้ที่ 90 % เป็นการค้นหา และ 10 % เป็นการจองแล้วคืน การคืนของทันทีทำให้ของในระบบไม่หมด จึงวัดต้นทุนในสภาวะปกติ ไม่ใช่สภาวะของโกลั่มด

4 ชั้น บน config ที่เสนอจริง

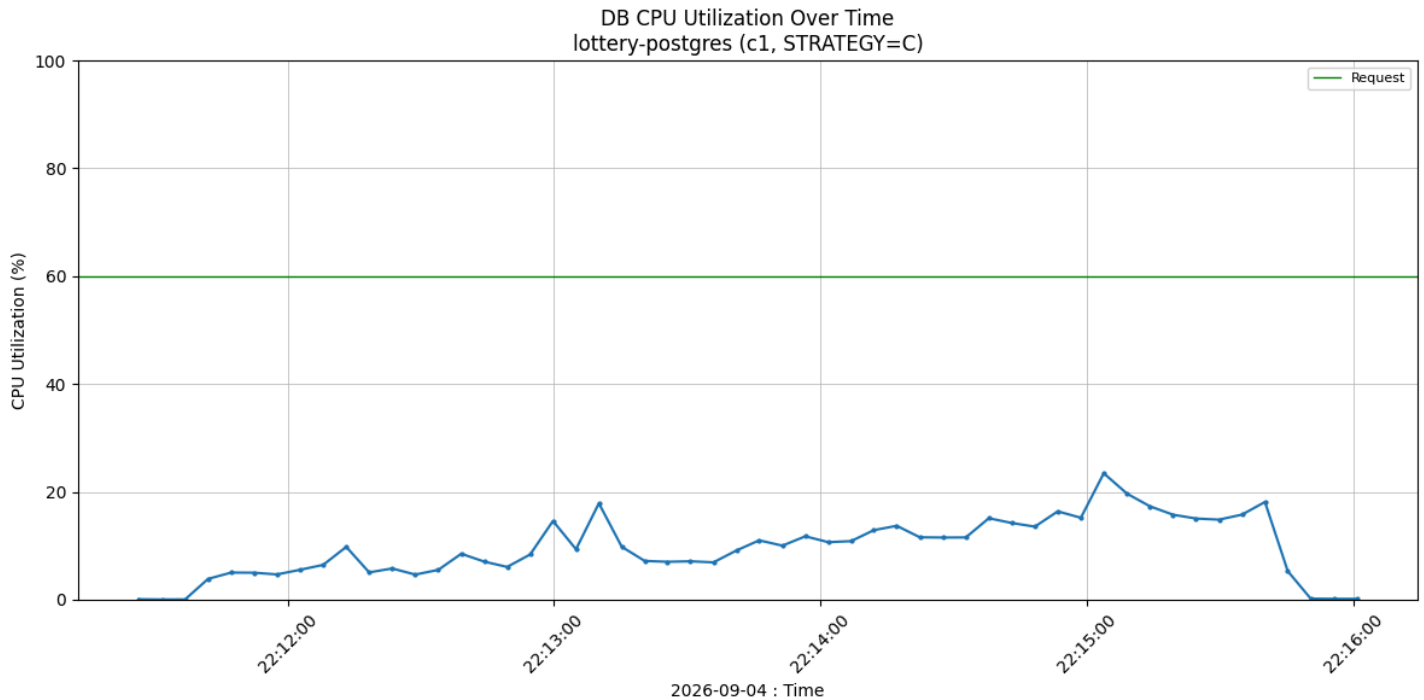
วัดบน API 4 pods และ Postgres 6 cores ตามที่เสนอ ทุกชั้นวัดเฉพาะช่วง 50 วินาทีที่อัตรานิ่งแล้ว ไม่นับ 10 วินาทีแรกที่กำลังได้ขึ้น

TPS	API ใช้	API % ของ LIMIT	DB ใช้	DB % ของ LIMIT	DB WORKING SET
1,000	112m	5.6 %	367m	6.1 %	1,014 MiB
2,000	230m	11.5 %	589m	9.8 %	1,270 MiB
3,000	302m	15.1 %	719m	12.0 %	1,373 MiB
4,000	441m	22.0 %	1,030m	17.2 %	1,439 MiB

latency ตลอดทั้งรอบ p95 = 1.55 ms และไม่มี request สัมเหลวเลยสักครั้ง



CPU ของ API เป็นชั้นบันได 4 ชั้นชัดเจน แต่ละชั้นคือ 1,000 TPS ที่เพิ่มขึ้น ความสูงของชั้นที่เท่ากันคือหลักฐานตรงๆ ว่าต้นทุนต่อ TPS คงที่



Postgres ก็เป็นชั้นบันไดเหมือนกัน แต่เริ่มจากฐานที่ไม่ใช่ศูนย์ เพราะมีงานพื้นหลังของ database เองอยู่ราว 146m ตลอดเวลา

DOCS/EVIDENCE/C1-CAPACITY/10-K6.LOG

✓ status 200

checks.....: 100.00% ✓ 579842 x 0

data_received.....: 403 MB 1.7 MB/s

data_sent.....: 96 MB 398 kB/s

x dropped_iterations.....: 153 0.637491/s

running (4m00.0s), 0000/0579 VUs, 550838 complete and 0 interrupted iterations

THRESHOLD ที่ไม่ผ่าน และทำไมไม่เป็นไร

dropped_iterations เท่ากับ 153 ครั้ง แปลว่าช่วงชั้นบนสุด k6 ตามอัตราที่ส่งไม่ทันอยู่หน่วยน้อย คิดเป็น **0.026 %** ของทั้งหมด ซึ่งเป็นข้อจำกัดของตัว k6 บนเครื่องนี้เอง ไม่ใช่ของระบบที่ทดสอบ เพราะฝั่ง server ไม่มี request ล้มเหลวเลยแม้แต่ครั้งเดียว และจำนวน request ที่ได้จริงยังตรงกับที่ตั้งไว้ถึง 99.97 %

แยกงานพื้นหลังออกจากต้นทุนต่อ request

ถ้าเอา CPU ทั้งหมดหารด้วย TPS ตรงๆ จะได้ตัวเลขที่ลดลงเรื่อยๆ ตามโหลดที่เพิ่ม เพราะมีงานพื้นหลังที่คงที่ปนอยู่ ต้องแยกสองส่วนนี้ออกจากกันก่อน โดยลากเส้นตรงผ่านจุดที่ต่ำสุดกับสูงสุด

CPU ที่ใช้ = งานพื้นหลัง + ต้นทุนต่อ request x TPS

API = 1.9m + 0.1098 m/TPS ที่ 10,000 TPS -> **1,099m**

DB = 146.1m + 0.2209 m/TPS ที่ 10,000 TPS -> **2,355m**

ตรวจเพดาน 60 %

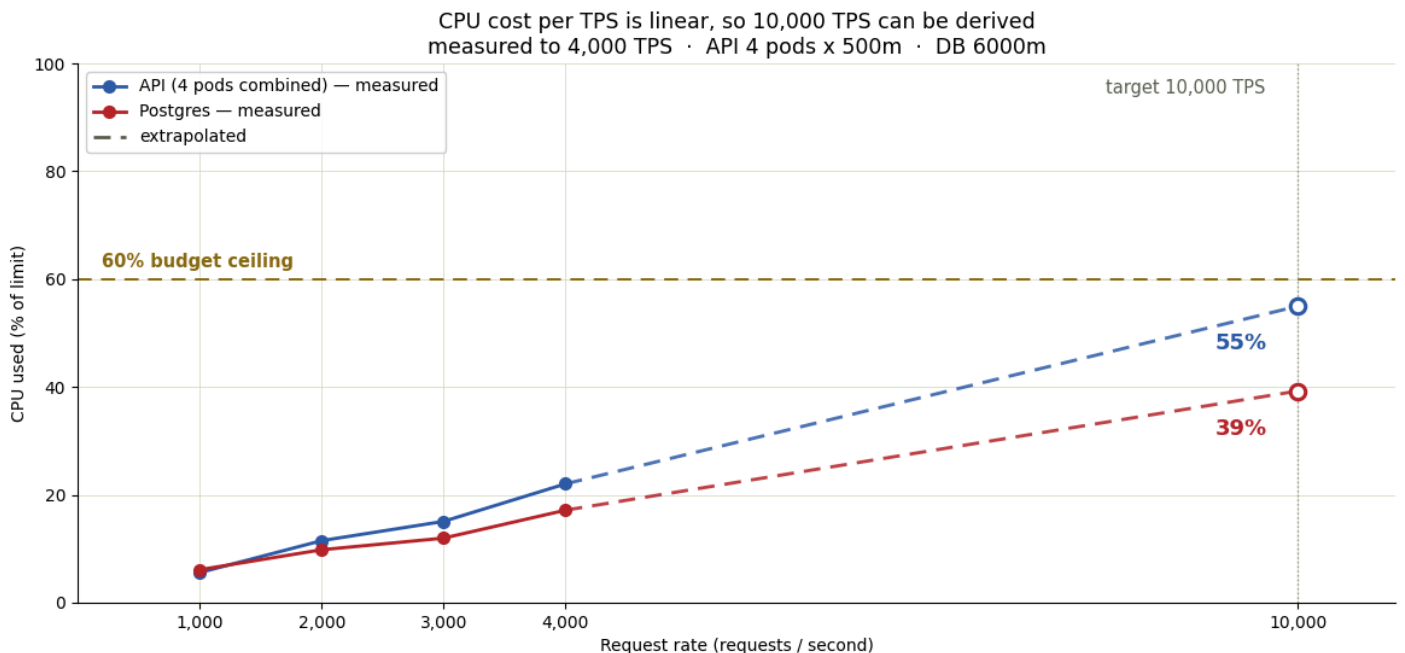
API 1,099m / (4 pods x 500m) = **55.0 %** ผ่าน

DB 2,355m / 6,000m = **39.3 %** ผ่าน

limit ขั้นต่ำที่ยังผ่านเพดาน

API 1,099m / 0.60 = 1,832m -> ต้องมีอย่างน้อย 4 pods x 500m = 2,000m

DB 2,355m / 0.60 = 3,926m -> ต้องมีอย่างน้อย 4,000m (จะได้ 58.9 % ซึ่งขีดเพดานเกินไป)



เส้นทึบคือของจริง เส้นประคือการคำนวณต่อ เส้นประแนวนอนคือเพดาน 60 % ที่ 10,000 TPS ทั้งสองเส้นยังอยู่ใต้เพดาน โดย API อยู่ที่ 55 % และ Postgres อยู่ที่ 39 % แกนตั้งเป็นเปอร์เซ็นต์ของ limit ของแต่ละตัว จึงเทียบกับเส้นงบเดียวกันได้ทั้งคู่

ทำไม API ถึงเป็นตัวที่ขีตเพดานมากกว่า ทั้งที่ database มักเป็นคอขวดในระบบทั่วไป คำตอบอยู่ใน รายงานผลทดสอบชุดก่อน คือ Strategy C ย้ายงานหนักออกจาก database ไปแล้ว งานที่เหลือของ database ต่อหนึ่ง request คือการอ่าน index ไม่กี่ครั้ง ส่วน API ต้องทำงาน HTTP, JSON และ connection pool ซึ่งกลายเป็นส่วนที่แพงกว่า

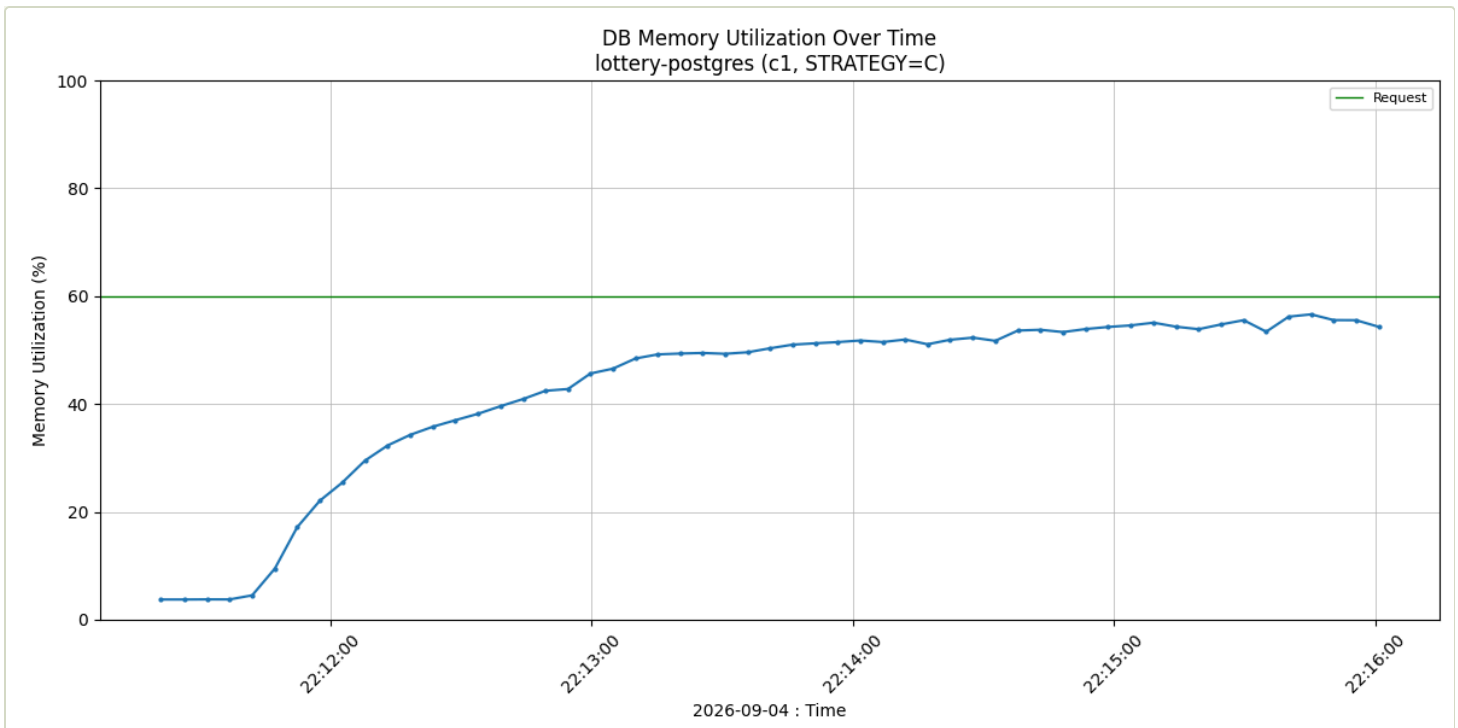
05 ข้อจำกัดอื่น

สามอย่างที่โมเดล CPU ไม่ครอบคลุม

1. Memory ของ database ไม่ได้โตเป็นเส้นตรง

ถ้าใช้วิธีลากเส้นตรงกับ memory จะได้คำตอบที่ผิด เพราะ memory ของ Postgres ไม่ได้โตตามจำนวน request แต่โตตาม *ปริมาณข้อมูลที่ถูกดึงเข้า cache* ซึ่งจะอึดตัวเมื่อข้อมูลร้อนถูก cache ครบแล้ว ดูจาก ส่วนต่างของแต่ละชั้นจะเห็นว่ามันลดลงเรื่อยๆ

working set ต่อชั้น	1,014	->	1,270	->	1,373	->	1,439 MiB
ส่วนต่าง			+256		+103		+66 (ลดลงราวครึ่งหนึ่งทุกชั้น)
คาดว่าอึดตัวที่	~1,510 MiB						ไม่ใช่การลากเส้นตรงซึ่งจะได้ราว 2,400 MiB
ที่ limit 2,560Mi	->		59.0 %				ผ่านแบบเฉียดฉิว
ที่ limit 3Gi	->		49.2 %				ผ่านสบาย <- เลือกอันนี้



memory ได้ขึ้นแล้วเริ่มแบน รูปทรงนี้คือการเติม cache จนอิม ไม่ใช่ memory leak ถ้าเป็น leak เส้นจะไต่ขึ้นเรื่อยๆ ในอัตราคงที่ โดยไม่มีที่ท่าจะแบน

2. Connection pool

จำนวน connection ที่ต้องใช้พร้อมกันคำนวณจาก *TPS คูณเวลาที่ใช้ต่อ request* ไม่ได้ขึ้นกับจำนวนผู้ใช้

เวลาเฉลี่ยฝั่ง server = 1.19 ms

ที่ 10,000 TPS -> $10,000 \times 0.00119 = 11.9$ connection ที่ต้องใช้พร้อมกัน

ที่มีอยู่ -> 4 pods x pgxpool 10 = 40 connection เหลือเพื่อ

max_connections ของ Postgres ตั้งไว้ 100

3. ขนาดของ node

รวม limit ทั้งหมดของ config ที่เสนอคือ CPU 8,000m และ memory 3,584Mi ในการวัดครั้งนี้ทำบน docker-desktop ที่มี 14 cores แต่มี memory ให้ Kubernetes ใช้ได้เพียง 3.73GiB จึงต้องบีบ database ลงเหลือ 2,560Mi ตอนทดสอบ ซึ่งยังผ่านเพดานที่ 56.2 % บน **production** ควรใช้ **3Gi** ตามที่เสนอ เพื่อให้มีระยะห่างจากเพดานมากกว่านี้

ขยายทางไหนก่อน

เป้าหมาย	API PODS	API ใช้	DB CPU LIMIT	DB ใช้
10,000 TPS ที่เสนอ	4	55 %	6,000m	39 %
15,000 TPS	6	55 %	6,000m	57 %
20,000 TPS	8	55 %	8,000m	56 %
ตัวเลขจากสูตรเดียวกัน ยังไม่ได้วัดจริง ใช้เป็นแนวทางวางแผนเท่านั้น				

ฝั่ง API ขยายง่ายเพราะไม่มี state เพิ่ม pod แล้วได้ throughput เพิ่มตามสัดส่วน ส่วนฝั่ง database จะไปชนเพดานของเครื่องเดียวก่อน จุดนั้นค่อยพิจารณา read replica สำหรับ `/v1/search` ซึ่งเป็น read อย่างเดียว โดยให้ `/v1/allocate` ยังยิงเข้า primary เท่านั้น เพราะ `SKIP LOCKED` ต้องการความถูกต้องระดับ transaction

อย่าเพิ่ง SCALE UP API POD

การเพิ่ม CPU ให้ API pod เดียวจาก 500m เป็น 1000m จะแทบไม่ช่วยอะไร เพราะตั้ง `GOMAXPROCS=1` ไว้ Go จะใช้ได้แค่ thread เดียวอยู่ดี ถ้าจะขยายแนวตั้งต้องเพิ่ม `GOMAXPROCS` ตามด้วย แต่การเพิ่มจำนวน pod ง่ายกว่า และได้ความทนทานเพิ่มจากการมีหลาย instance ด้วย

เชื่อมกลับไปที่คำถามทั้ง 4 ข้อ

คำถามจากโจทย์	คำตอบ
ใช้ database อะไร	PostgreSQL ตัวเดียว 6 cores / 3Gi รับ 10,000 TPS ได้ที่ 39 % ของ limit ไม่ต้องมี Redis ไม่ต้องมี distributed lock และไม่ต้อง replication
algorithm และ index	Strategy C คือค้นเลขที่เข้า pattern จากตาราง 1 ล้านแถวก่อน โดยแปลงหลักหน้าที่ล็อกไว้เป็นช่วงบน primary key แล้ว join เข้าตารางสลาก ทำให้ต้นทุนฝั่ง database เหลือเพียง 0.22 m-core ต่อ 1 TPS
กันผลลัพธ์ซ้ำ	<code>FOR UPDATE SKIP LOCKED</code> ในคำสั่งเดียว พิสูจน์แล้วด้วยการจอง 351,481 ใบ ภายในได้ 200 คนแย่ง pattern เดียวกัน ไม่มีตัวซ้ำแม้แต่ใบเดียว
รองรับ 10M+ และ โหลดสูง	ต้นทุนการค้นหาไม่ขึ้นกับจำนวนสลาก และต้นทุนต่อ TPS เป็นเส้นตรง จึงวางแผนขยายได้ด้วยสูตร ที่ 10,000 TPS ใช้ API 4 pods กับ Postgres 1 ตัว โดยทั้งคู่อยู่ได้เพดาน 60 %

สิ่งที่ตัวเลขชุดนี้ยังไม่ได้พิสูจน์

- **10,000 TPS** มาจากการคำนวณ ไม่ได้จริง ค่าที่วัดสูงสุดคือ 4,000 TPS การลากเส้นต่อไปอีก 2.5 เท่าตั้งอยู่บนสมมติฐานว่ายังเป็นเส้นตรงอยู่ ซึ่งข้อมูล 4 จุดที่มีสนับสนุนสมมติฐานนี้ แต่ไม่ได้พิสูจน์ว่าจะจริงตลอดไป ก่อนขึ้น production ควรยืนยันจากเครื่องแยกที่ไม่ใช่เครื่องเดียวกับ cluster
- ยังไม่ได้ทดสอบสถานะของ **ไกล์หมด** รอบนี้คืนของทันทีทุกครั้ง ในของจริงถ้า pattern ยอดนิยมถูกจองไปเกือบหมด ต้นทุนต่อ request จะสูงขึ้นมาก ตามที่เห็นในรายงานชุดก่อนที่ latency ขึ้นจาก 74 ms เป็น 548 ms
- ทุกอย่างรันบน **node เดียว** ทั้ง API, database และตัวยิงโหลดแย่ง CPU ชุดเดียวกัน บน production ที่แยกเครื่องกัน ตัวเลขน่าจะดีกว่านี้
- ไม่ได้ทดสอบตอน **pod ล่มหรือ rolling update** ซึ่งเป็นสถานะที่ capacity ลดลงชั่วคราว ควรเผื่อ pod เพิ่มอีกอย่างน้อย 1 ตัวถ้าต้องการรองรับการอัปเดตแบบไม่มี downtime

ตัวเลขทั้งหมดมาจาก `docs/evidence/c1-capacity/` ซึ่งมี log ของ k6 แบบเต็ม กราฟทรัพยากร 10 รูป ข้อมูลดิบ CSV และไฟล์โมเดล `50-capacity.json` รันซ้ำได้ด้วย `scripts/scale.sh up` ตามด้วย `make capture-c1` แล้วคำนวณใหม่ด้วย `python3 scripts/capacity.py docs/evidence/c1-capacity 10000 0.60` กลับไปใช้ config เดิมของ PoC ได้ด้วย `scripts/scale.sh down`